

从房价预测到分子性质预测

AI药物设计 - 第3次课

李哲 副教授

药学院 412B

lizhe5@mail.sysu.edu.cn

开始探索预测的奥秘 →

2025/12/12



今天的学习目标

-  **理解线性关系** - 掌握如何用数学模型描述现实世界的关系
-  **学会预测思维** - 从已知数据推断未知结果的基本方法
-  **连接两个世界** - 将房价预测的逻辑应用到分子性质预测
-  **数字化分子** - 理解计算机如何"看懂"分子结构
-  **建立直觉** - 为后续深入学习AI药物设计打下基础

核心问题：预测房价的数学逻辑，如何用来预测分子性质？

第一节：线性回归入门

从熟悉的房价预测开始

为什么从房价开始？

药物研发的本质问题

- 一个新分子能否成为好药？
- 它的毒性有多大？
- 口服吸收效果如何？

房价预测的本质问题

- 这套房子值多少钱？
- 面积大小如何影响价格？
- 地段好坏怎么量化？

共同点 🤔

- 都是从**已知特征**预测**未知结果**
- 都需要找到**特征与结果的关系**
- 都希望预测**尽可能准确**

💡 **关键洞察：**预测房价的数学逻辑，完全可以用来预测分子性质！

小明买房的困惑

 房子编号	 面积(m ²)	 价格(万元)
--	---	--

1	50	150
---	----	-----

2	65	180
---	----	-----

3	80	220
---	----	-----

4	95	250
---	----	-----

5	110	290
---	-----	-----

 房子编号	 面积(m ²)	 价格(万元)
--	---	--

6	45	140
---	----	-----

7	75	200
---	----	-----

8	88	240
---	----	-----

9	102	275
---	-----	-----

10	120	310
----	-----	-----

如果小明看中一套 **85m²** 的房子， 大概需要多少钱？

第一步：画图观察规律

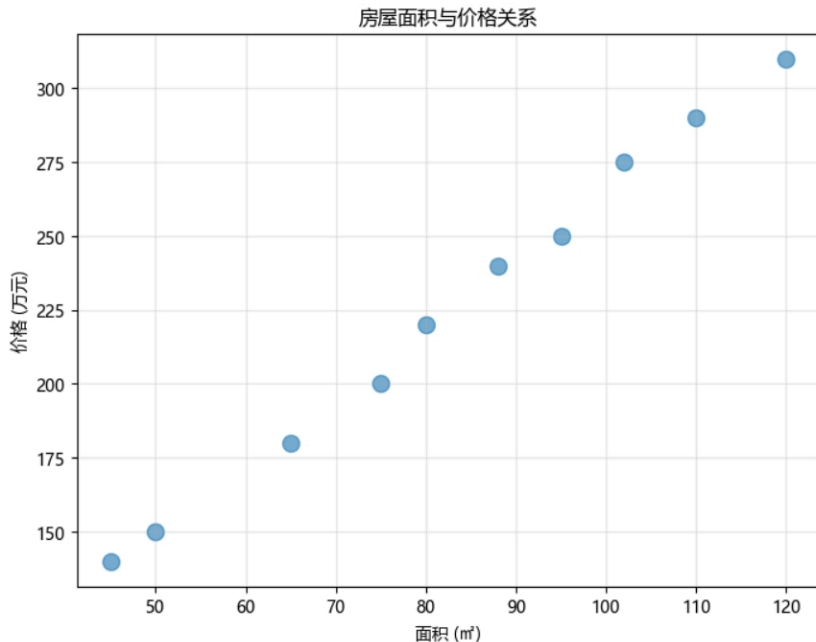
想象我们把这些数据画成散点图：

- **横轴**：房屋面积 (m^2)
- **纵轴**：房屋价格 (万元)

观察发现

- 面积越大，价格越高 
- 这些点几乎排成一条直线 
- 存在明显的**线性关系**

 **直觉理解**：房屋面积是决定价格的主要因素



什么是线性关系？

日常生活中的线性关系 🌟

1. 买菜 🍎：苹果5元/斤，买 x 斤花 $5x$ 元
2. 打车 🚗：起步价10元，每公里3元
3. 手机话费 📠：月租30元，每分钟0.2元

数学表达： $y = wx + b$

- y ：我们要预测的结果（房价）
- x ：我们已知的特征（面积）
- w ：斜率， x 每增加1， y 增加多少
- b ：截距，当 $x=0$ 时 y 的值

$y = wx + b$ 就是一条直线的方程！

房价公式的参数解读

$$w \text{ (斜率)} = 2.5 \text{ 万元/m}^2$$

实际意义：每增加 1m^2 ，房价增加2.5万元

- 这就是**每平方米单价**！💰
- w 值越大，房价涨得越快 📈

$$b \text{ (截距)} = 25 \text{ 万元}$$

实际意义：即使面积为0，仍有25万元基础价格

- 土地价值、基础设施、品牌溢价等 🏠
- 不管房子多大，都有这个“底价” 💎

完整公式 🧮

$$\text{房价} = 2.5 \times \text{面积} + 25$$

互动练习：计算房价

用公式 $\text{房价} = 2.5 \times \text{面积} + 25$ 来预测：

练习1：85m²的房子多少钱？

$$\text{房价} = 2.5 \times 85 + 25 = 212.5 + 25 = \mathbf{237.5\text{万元}}$$

练习2：你来算算看！

90m²的房子
房价 = ?

60m²的房子
房价 = ?

130m²的房子
房价 = ?

关键问题：怎么找到最佳的w和b？

从一套房子的误差说起

- 我们的模型：**预测价格** $\hat{y} = w \times x + b$
- 第 i 套房子：真实价格是 y_i ，预测价格是 $\hat{y}_i$
- 对这一套房子，**误差**：
 $e_i = y_i - \hat{y}_i$ （差了多少万元）

多套房子时怎么衡量“好”？

- 有 10 套房子，就有 10 个误差 $e_1, e_2, \dots, e_{10}$
- 直接把 e_i 相加会有正有负，**会互相抵消**
- 所以我们看 **误差的平方**： e_i^2 （永远是正数）
- 定义一个“总误差”函数：
$$L(w, b) = \sum (y_i - (w x_i + b))^2$$

最小二乘法 (Least Squares):

在所有可能的 w 、 b 里面，找到让
 $L(w, b) = \sum (y_i - (w x_i + b))^2$ **最小** 的那一对。

 也就是让所有点到直线的总“偏离程度”最小。

Python快速实现

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 房屋数据准备
area = np.array([50, 65, 80, 95, 110, 145, 75, 88, 102, 12
price = np.array([150, 180, 220, 250, 290, 140, 200, 240,

# 创建并训练线性回归模型
model = LinearRegression()
model.fit(area, price)

# 获取模型参数
w = model.coef_[0]      # 斜率（每m²价格）
b = model.intercept_    # 截距（基础价格）

# 预测85m²房价
predicted = model.predict([[85]])
print(f"预测价格: {predicted[0]:.1f}万元")
print(f"斜率w = {w:.2f}, 截距b = {b:.2f}")
```

运行结果

预测价格: 230.2万元
斜率 $w = 2.33$ 万元/m²
截距 $b = 32.04$ 万元

结果解释:

-  每平米价值: 2.33 万元
-  基础成本: 32.04 万元

扩展：多元线性回归预览

单特征 vs 多特征

单特征（今天）：

- 房价 = $f(\text{面积})$
- $y = wx + b$

多特征（下节课）：

- 房价 = $f(\text{面积}, \text{楼层}, \text{地段}, \text{朝向}\dots)$
- $y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$

药物设计中的应用

单描述符：

- 活性 = $f(\text{分子量})$

多描述符：

- 活性 = $f(\text{MW}, \text{LogP}, \text{TPSA}, \text{HBD}, \text{HBA}\dots)$



预告：下节课我们将学习如何处理多个特征！

第一节小结

线性回归的核心要点

线性回归：从数据到预测

1 核心概念

- **线性关系：** $y = wx + b$
- **参数意义：** w 是变化率， b 是基础值
- **目标：**预测误差最小

2 关键步骤

1. 收集数据（特征+结果）
2. 观察关系（画图分析）
3. 建立模型（确定公式）
4. 优化参数（最小二乘法）
5. 预测新值（代入公式）

3 实际应用

- 任何线性关系的预测问题
- 为复杂机器学习打基础
- **接下来：**应用到分子性质预测！

第二节：分子的数字化表示

从房子特征到分子特征

思维转换：从房价到药价

房价预测

- **输入特征**：面积（数字）
- **输出结果**：价格（数字）
- **关系**：线性关系

分子性质预测

- **输入特征**：分子描述符（数字）
- **输出结果**：生物活性（数字）
- **关系**：可能线性，也可能复杂

关键问题

分子不是数字，计算机怎么理解分子？

 **答案**：将分子转换为数字特征（分子描述符）

类比思考：房子 vs 分子

房子的特征描述

- 面积：100m² 
- 房间数：3室2厅 
- 楼层：18层中的8层 
- 朝向：南北通透 
- 年龄：建成5年 

分子的特征描述

- 分子量：180.16 g/mol 
- 脂溶性：LogP = 1.38 
- 极性：TPSA = 63.6 Å² 
- 氢键：3个供体，4个受体 
- 结构：包含苯环和羧基 

 发现：都可以用数字和类别来描述！

为什么计算机需要分子描述符？



计算机的限制

- 只能处理数字
- 不能直接"看懂"分子结构
- 需要数字化转换

类比：AI识别人脸

- 不存储整张照片
- 提取关键特征：眼距、鼻型等
- 用数字特征识别

分子描述符作用

- 简化复杂分子结构
- 反映药学性质
- 支持计算机预测

让计算机"看懂"分子：从结构到SMILES

分子结构的挑战

- 3D立体结构复杂
- 原子间连接关系多样
- 难以直接输入计算机

SMILES：分子的"文字"表达

SMILES = Simplified Molecular Input Line Entry System

将分子结构转换为一串字符，就像写化学式一样简单！

实例对比

阿司匹林：

- 化学式：C₉H₈O₄
- SMILES: CC(=O)Oc1ccccc1C(=O)O

咖啡因：

- 化学式：C₈H₁₀N₄O₂
- SMILES: CN1C=NC2=C1C(=O)N(C(=O)N2C)C

 **优势：** 计算机可以像处理文本一样处理分子！

化学信息学工具：RDKit简介

什么是RDKit?

- 开源化学信息学工具包 
- Python/C++编写，免费使用 
- 分子处理的瑞士军刀 
- 全球制药公司广泛使用 

核心功能

- 分子结构读取与转换
- 描述符计算与分析
- 分子相似性比较
- 化学反应处理
- 3D构象生成

安装RDKit

```
# 使用conda安装（推荐）  
conda install -c conda-forge rdkit  
  
# 或使用pip安装  
pip install rdkit
```

基本导入

```
from rdkit import Chem  
from rdkit.Chem import Descriptors  
from rdkit.Chem import Draw  
from rdkit.Chem import rdMolDescriptors  
import pandas as pd  
import matplotlib.pyplot as plt
```

重要的分子描述符

1. 分子量 (MW)

定义：所有原子质量的总和

药学意义：

- $MW < 200$ ：可能缺乏特异性
- $MW > 500$ ：膜透性下降
- **Lipinski最适范围： ≤ 500 Da** 

2. 脂水分配系数 (LogP)

定义：分子在油水两相中的分配比例

- $\text{LogP} > 0$ ：脂溶性强
- $\text{LogP} < 0$ ：水溶性强
- **理想范围：0-3** 

3. 极性表面积 (TPSA)

定义：分子表面极性原子所占面积

药学意义：

- $\text{TPSA} < 140 \text{ \AA}^2$ ：易被肠道吸收 
- $\text{TPSA} < 60 \text{ \AA}^2$ ：可能进入大脑 

4. 氢键数量 (HBD/HBA)

氢键供体 (HBD)：能提供氢原子的基团 **氢键受体 (HBA)**：能接受氢原子的基团

Lipinski五规则要求：

- $\text{HBD} \leq 5$
- $\text{HBA} \leq 10$

RDKit实战1：分子的读取与创建

从SMILES创建分子对象

```
from rdkit import Chem

# 📄 创建阿司匹林分子对象
aspirin_smiles = "CC(=O)Oc1ccccc1C(=O)O"
aspirin = Chem.MolFromSmiles(aspirin_smiles)

# ☕ 创建咖啡因分子对象
caffeine_smiles = "CN1C=NC2=C1C(=O)N(C(=O)N2C)C"
caffeine = Chem.MolFromSmiles(caffeine_smiles)

# ✅ 验证分子对象创建状态
print(f"阿司匹林分子对象: {aspirin}")
print(f"咖啡因分子对象: {caffeine}")
```

输出结果

```
阿司匹林分子对象: <rdkit.Chem.rdchem.Mol object>
咖啡因分子对象: <rdkit.Chem.rdchem.Mol object>
```

验证分子有效性

```
# 🔍 验证分子SMILES有效性
if aspirin is not None:
    print("✅ 阿司匹林SMILES解析成功")
else:
    print("❌ 阿司匹林SMILES解析失败")

if caffeine is not None:
    print("✅ 咖啡因SMILES解析成功")
else:
    print("❌ 咖啡因SMILES解析失败")
```

RDKit实战2：分子可视化

单分子渲染

```
from rdkit.Chem import Draw
import matplotlib.pyplot as plt
```

```
# 🎨 单个分子可视化
```

```
img = Draw.MolToImage(aspirin, size=(300, 300))
img.show() # 显示阿司匹林结构图
```

批量分子对比

```
# 📋 准备分子数据
```

```
molecules = [aspirin, caffeine]
legends = ['阿司匹林', '咖啡因']
```

```
# 🎨 网格布局可视化
```

```
img = Draw.MolsToGridImage(molecules, molsPerRow=2,
                             subImgSize=(250, 250), legends=
```

```
# 💾 保存并显示图片
```

```
img.save('molecules_comparison.png')
img.show()
```

可视化的价值

- **直观理解：** 分子结构一目了然
- **快速比较：** 多分子并列对比
- **科研展示：** 论文图表制作
- **教学工具：** 化学教育利器

氢原子处理：为什么重要？



😬 为什么需要氢原子处理？

SMILES默认隐式氢：CCO表示乙醇，氢原子被省略

3D构象生成：需要显式氢原子位置

精确计算：氢键、表面积等需要氢

药物分析：氢键供体计算必需

显式添加氢原子

```
# 🧪 创建不含氢的分子
mol_no_h = Chem.MolFromSmiles("CCO")

# ➕ 添加显式氢原子
mol_with_h = Chem.AddHs(mol_no_h)

# 📊 比较原子数量
print(f"不含氢: {mol_no_h.GetNumAtoms()}个原子")
print(f"含氢: {mol_with_h.GetNumAtoms()}个原子")
# 输出: 不含氢: 3个原子, 含氢: 9个原子
```

分子标准化处理 🖌️

```
# 🌸 分子标准化（去除盐等）
from rdkit.Chem import SaltRemover
remover = SaltRemover.SaltRemover()

# 🍷 移除常见盐类
clean_mol = remover.StripMol(mol_with_salt)
```


分子基本信息提取

结构统计信息

```
# 🧪 阿司匹林分子对象
mol = Chem.MolFromSmiles("CC(=O)Oc1ccccc1C(=O)O")

# 📊 原子和键的数量统计
print(f"原子数: {mol.GetNumAtoms()}")
print(f"键数: {mol.GetNumBonds()}")
print(f"环数: {mol.GetRingInfo().NumRings()}")

# 🧬 获取分子式
from rdkit.Chem.rdMolDescriptors import CalcMolFormula
formula = CalcMolFormula(mol)
print(f"分子式: {formula}")
```

实际输出结果

原子数: 13
键数: 13
环数: 1
分子式: C₉H₈O₄

结果解读

 **原子数 = 键数**

说明结构为非环链状加一个环

 **环数 = 1**

确认含有苯环结构

 **分子式 C₉H₈O₄**

与阿司匹林理论值一致

 **分子量**

$9 \times 12 + 8 \times 1 + 4 \times 16 = 180$

第三节：RDKit分子描述符实战

从理论到代码实现

RDKit实战3：物理化学性质描述符

```
1 # 📊 计算核心物理化学性质
2 mw = Descriptors.MolWt(aspirin)           # 分子量
3 logp = Descriptors.MolLogP(aspirin)       # 脂水分配系数
4 tpsa = Descriptors.TPSA(aspirin)         # 极性表面积
5 hbd = Descriptors.NumHDonors(aspirin)     # 氢键供体数
6 hba = Descriptors.NumHAcceptors(aspirin)  # 氢键受体数
7
8 # 📐 计算结构特征描述符
9 heavy_atoms = Descriptors.HeavyAtomCount(aspirin) # 重原子数
10 rotatable_bonds = Descriptors.NumRotatableBonds(aspirin) # 可旋转键数
11 aromatic_rings = Descriptors.NumAromaticRings(aspirin) # 芳香环数
12
13 # 🖨️ 格式化输出结果
14 print(f"💰 分子量: {mw:.2f} g/mol")
15 print(f"💜 LogP: {logp:.2f}")
16 print(f"⚡ TPSA: {tpsa:.2f} Å")
17 print(f"🔗 氢键供体: {hbd} 个")
18 print(f"🔗 氢键受体: {hba} 个")
19 print(f"🌀 重原子数: {heavy_atoms} 个")
20 print(f"🔗 可旋转键: {rotatable_bonds} 个")
21 print(f"🕒 芳香环: {aromatic_rings} 个")
```

描述符计算结果分析

输出结果

分子量: 180.16 g/mol
 LogP: 1.19
 TPSA: 63.60 Å²
 氢键供体: 1
 氢键受体: 4
 重原子数: 13
 可旋转键: 3
 芳香环: 1

Lipinski五规则验证

```
#  Lipinski五规则验证函数
def check_lipinski_five_rules(mol):
    """验证分子是否符合Lipinski五规则"""
    mw = Descriptors.MolWt(mol)
    logp = Descriptors.MolLogP(mol)
    hbd = Descriptors.NumHDonors(mol)
    hba = Descriptors.NumHAcceptors(mol)
    rotbonds = Descriptors.NumRotatableBonds(mol)

    #  完整五规则检查
    rules = {
        '分子量 ≤ 500': mw <= 500,
        'LogP ≤ 5': logp <= 5,
        '氢键供体 ≤ 5': hbd <= 5,
        '氢键受体 ≤ 10': hba <= 10,
        '可旋转键 ≤ 10': rotbonds <= 10
    }

    passed_count = sum(rules.values())

    #  详细结果输出
    print(f" Lipinski五规则验证结果:")
```

RDKit实战4：分子指纹描述符

Morgan指纹 (ECFP)

```
from rdkit.Chem import rdMolDescriptors
from rdkit import DataStructs

# 生成Morgan指纹 (半径=2, 2048位)
morgan_fp = rdMolDescriptors.GetMorganFingerprintAsBitVec(
    aspirin, radius=2, nBits=2048)

# 转换为Python列表 (前10位示例)
morgan_bits = list(morgan_fp.ToBitString())
print(f"Morgan指纹长度: {len(morgan_bits)}")
print(f"前10位: {morgan_bits[:10]}")

# 计算分子相似性
caffeine = Chem.MolFromSmiles("CN1C=NC2=C1C(=O)N(C(=O)N2C)C")
caffeine_fp = rdMolDescriptors.GetMorganFingerprintAsBitVec(
    caffeine, radius=2, nBits=2048)

# Tanimoto相似性
similarity = DataStructs.TanimotoSimilarity(morgan_fp, caffeine_fp)
print(f"阿司匹林与咖啡因相似性: {similarity:.3f}")
```

MACCS分子指纹

```
from rdkit.Chem import MACCSKeys

# 生成MACCS指纹 (166位固定长度)
maccs_fp = MACCSKeys.GenMACCSKeys(aspirin)

print(f"MACCS指纹长度: {maccs_fp.GetNumBits()}")
print(f"激活位数: {maccs_fp.GetNumOnBits()}")

# 获取激活的位索引
on_bits = list(maccs_fp.GetOnBits())
print(f"激活位索引: {on_bits[:10]}...")

# 与咖啡因比较
caffeine_maccs = MACCSKeys.GenMACCSKeys(caffeine)
maccs_similarity = DataStructs.TanimotoSimilarity(maccs_fp, caffeine_maccs)
print(f"MACCS相似性: {maccs_similarity:.3f}")
```

RDKit实战5：分子筛选：PAINS过滤器

什么是PAINS?

PAINS = Pan-Assay Interference Compounds

- **定义**：泛检测干扰化合物
- **问题**：在多种生物检测中产生假阳性
- **机制**：非特异性结合、聚集形成等
- **影响**：浪费研发资源，误导药物设计

为什么重要？

- 避免假阳性先导化合物
- 提高药物研发成功率
- 节省时间和资金成本

RDKit中的PAINS过滤

```
from rdkit.Chem import FilterCatalog
from rdkit.Chem.FilterCatalog import FilterCatalogParams

# 创建PAINS过滤器
params = FilterCatalogParams()
params.AddCatalog(FilterCatalogParams.FilterCatalogs.PAIN
filter_catalog = FilterCatalog.FilterCatalog(params)

# 检测PAINS化合物
def check_pains(mol, name):
    if filter_catalog.HasMatch(mol):
        matches = filter_catalog.GetMatches(mol)
        print(f"⚠️ {name} 包含PAINS子结构")
        for match in matches:
            print(f"    - {match.GetDescription()}")
        return False
    else:
        print(f"✅ {name} 通过PAINS筛选")
        return True
```

示例：检测阿司匹林

阿司匹林案例分析回顾

RDKit计算结果

使用RDKit计算的实际值

```
aspirin = Chem.MolFromSmiles("CC(=O)Oc1ccccc1C(=O)O")
```

分子量: 180.16 g/mol 

LogP: 1.19 

TPSA: 63.60 Å² 

氢键供体: 1个 

氢键受体: 4个 

可旋转键: 3个 

芳香环: 1个 

与理论值对比

- 分子量: 180.16 Da (与理论值一致)
- LogP: 1.19 (文献范围1.19-1.38)
- TPSA: 63.60 Å² (精确匹配文献值)
- Lipinski五规则: 5/5完全通过

药学意义验证

 MW < 500: 符合口服药物要求

 LogP适中: 良好的膜透性

 TPSA < 140: 利于肠道吸收

 所有规则通过: 优良的类药性

 **结论:** 阿司匹林是理想的口服药物

第四节：综合实战与应用

从简单预测到分子设计

构建分子性质预测器

项目目标

使用RDKit和机器学习构建一个预测分子溶解度的模型：

1. **数据准备**：收集分子SMILES和溶解度数据
2. **特征工程**：计算分子描述符
3. **模型训练**：线性回归预测溶解度
4. **模型评估**：测试集性能评估

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
```

示例数据（实际项目中使用真实数据库）

```
solubility_data = {
    'SMILES': [
        'CCO', # 乙醇
        'CCCCCCC', # 辛烷
        'O', # 水
        'c1ccccc1', # 苯
        'CC(=O)O', # 乙酸
    ],
    'LogS': [-0.24, -5.15, 0.0, -2.13, -0.17]
}
```

```
def calculate_features(smiles_list):
    """计算分子描述符特征矩阵"""
    features = []
    for smiles in smiles_list:
        mol = Chem.MolFromSmiles(smiles)
        if mol:
            feature_vector = [
```

互动练习：哪个药物最适合口服？

给定以下三个分子的SMILES，判断哪个最适合口服给药：

选项A

CCCCCCCCCCCCCCCC
(十六烷)

选项B

CC(=O)Nc1ccc(O)cc1
(对乙酰氨基酚)

选项C

O=C(O)CCCCCCCCCCCCC
(棕榈酸)



提示：考虑Lipinski规则的各项指标

答案揭晓

选项A ❌

MW = 226 ✓

LogP ≈ 8.8 ✗

太油腻，无法吸收

选项B 

MW = 151 ✓

LogP ≈ 0.27 ✓

所有指标都符合!

选项C ❌

MW = 256 ✓

LogP ≈ 7.17 ✗

脂肪酸难吸收



正确答案：选项B (对乙酰氨基酚)

课堂练习

 **练习A：基础操作**
维生素C的描述符

难度: ★★☆☆

 **练习B：相似性分析**
比较咖啡因与茶碱的相似性

难度: ★★★

 **练习C：分子筛选**
筛选符合Lipinski规则的分子

难度: ★★★★★

全课总结

从理论到实践的完整闭环

今天课程的核心价值

理论基础

1. **线性回归逻辑**：从房价预测理解ML基础
2. **分子表示理论**：SMILES与分子描述符
3. **描述符体系**：物化性质、指纹、拓扑描述符
4. **类药性评估**：Lipinski规则及PAINS过滤

实践技能

1. **RDKit熟练使用**：从分子读取到批量处理
2. **描述符计算**：各类描述符完整计算流程
3. **数据分析能力**：相似性分析、筛选管线
4. **代码实现**：从单分子到数据库级别操作

 **关键成就**：你现在具备了进行实际AIDD项目的基础工具和思维框架！

从简单到复杂的学习路径

今天的基础：单分子描述符计算

1

掌握RDKit基本操作，理解分子数字化概念

下节课预期：多特征线性回归

2

使用多个描述符预测分子性质，特征选择与工程

未来方向：深度学习与分子生成

3

神经网络、图神经网络、分子生成模型